The MX Manifesto

MX is to machines what UX is to users. Every system that publishes or stores content now publishes to machines, under a contract written only for humans. MX is how that contract comes to hold for the machine too.

The contract written for humans

A CMS controls a publishing point, and with that comes an unwritten agreement with the data owner: “We will make your content visible to humans in a way that benefits you.” Machines now fall under it, whether anyone asked for them to or not. Vendors who fail to update it will lose trust.

Since the web began, every CMS, DAM, PIM, CRM, hosting platform, and cloud provider has run on that contract: publish the owner’s content to people, for the owner’s benefit. The carriers were HTML, PDF, images, feeds, and documents. The goals were human. Make a sale. Inform a citizen. Tell a story. Deliver a service. Governance, metadata, permissions, and lifecycle were all built around a human reader. Everyone understood the terms, and they worked.

The reader changed

Machines now read the same content: the website, the PDFs, the spreadsheets, the slide decks, the shared drives, the documentation, the policy files. They read files, not pages. They link files together, fill gaps by inference, and act on content nobody prepared for them. The contract has a second reader, and nobody has rewritten it.

The owner still believes the original terms hold: make my content visible to people, in a way that serves me. Reality has added a clause the owner never signed: make it interpretable and verifiable for the machine that reads it next. Owners will not ask for that clause by name. They will question the vendor’s competence when an agent gets it wrong.

This already happened, in the open

This is no forecast. The contract shifts on one side’s terms, and the publisher absorbs the change. Google narrowed FAQ rich results to a small set of sites and withdrew How-To results entirely. Pages that had done the structured-data work correctly lost the display overnight, with no notice anyone could negotiate. The party reading the markup changed the terms, and the sites that published it paid for the change. The machine-facing contract is already in force, and it is already one-sided.

Inference is the exposure

The old contract never required explicit structure, meaning, state, rights, or provenance, so machines infer them. Inference is how reasoning works, and it is not going away. The exposure is narrower, and real: ungoverned inference over unverified inputs. An agent left to guess a file’s status, its licence, or its origin pays for the guess, then acts on it. The cost arrives as a wrong answer, a leaked fact, or a citation that was never true.

Determinism, not inference

MX prefers determinism to inference. Same input, same result, every time. A file that states its provenance, its status, and its terms leaves the machine nothing to guess. Two agents reading it reach the same answer, and so does the same agent tomorrow. Where a rule can do a check, a rule does it. Inference keeps its place, since reasoning over verified

inputs is what agents are for, but the inputs themselves should never be a guess. State it rather than leave it to inference. Record it rather than rely on memory. Prefer a result you can reproduce to one you can only explain.

Metadata as the engine

Most people think of metadata as documentation: data about data, a description of something else. Metadata can do more. It can tell a machine what to do next. This is MATEOS, Metadata As The Engine Of Application State. A field like `signatureExpires: 2027-06-30` is an instruction to every machine that reads the file: verify the signature, then check the date, and if today is past it, stop trusting the file. The metadata runs the system as well as describing it.

Here MX parts company with older metadata schemes. Each field tells a machine what to verify next, what conflicts exist, when to stop trusting, and where to find evidence. A `conflicts` field says: account for this bias when you evaluate the claim. A `refersToExternal` field grants permission: you may verify this claim against these independent sources. Together, the fields form the protocol the machine walks to reach a decision.

Governance enforced mechanically, through what each field means, runs the same way every time. Governance written as prose depends on someone remembering to apply it by hand. A field that makes a rule executable does work a rulebook that only describes the rule cannot. That is why MX is built the way it argues: as metadata protocols that make each machine step verifiable and the result auditable, rather than as documentation of how things ought to behave.

A guardrail you can argue with

The exposure widens the moment the machine stops reading and starts acting. A model that says something wrong is a problem for the page. An agent that does something wrong (calls a tool it should not, charges a card twice, emails the entire customer list) is an incident with a postmortem attached. Checks on the way in and the way out are necessary and never sufficient: injection caught at the boundary, schema validated on entry, output redacted before it leaves. The widest blast radius is still the action itself. Which tools the agent may touch. What a human must sign off before it runs, and what it may fire on its own. What gets rate limited, so a reasoning loop cannot become a thousand calls. What gets verified after execution, and not only authorised before it. Almost nobody writes tests around that layer, and it is the one that can burn the estate down.

The enforcement has to be deterministic. Ask one model to police another and the result is open to argument, and any argument you can make, an attacker can make too. If someone can talk their way past a guardrail, it guards nothing. Policy belongs in code, with a plain allow or deny, run the same way every time, leaving a record. This is the determinism rule again, applied to actions instead of inputs.

None of it is new. Idempotency, so a retried action does not charge the card twice. Circuit breakers, so one failing tool does not cascade through the whole flow. Audit logs you can replay, so a wrong outcome can be reconstructed exactly: what the agent decided, and why. Backpressure, so the system degrades instead of falling over. Backend distributed systems have leaned on these primitives for a decade, and they are now pointed at agents and called AI safety. Agentic work is an old discipline in a new jacket. The estates that will ship agents safely are the ones that already know how to operate untrusted, concurrent, side-effecting systems without setting the building alight, however clever their prompts. Far from slowing the work down, enforcement of this kind is what makes it safe enough to ship.

Built the way it argues

A standard is credible only if its authors run on it, and this repository is the working proof. It is self-describing: every file carries its own metadata, and the repository keeps machine-readable models of itself, including a cog registry, a routing registry, and a documentation graph. It is self-checking: where a rule can be a check, a check enforces it, so its claims are verified rather than asserted. It is also self-healing: it regenerates its indexes and repairs its own drift, but only when it can see all of itself. Healing from a partial view would heal wrong, so the repository acts only when it is whole enough

and declines when it is not. That is the discipline MX asks of any estate, turned inward. The detail is in the repository architecture reference.

The same discipline governs every working session. When a session surfaces a pre-existing warning alongside its own changes (a cog missing a required field, a test already red when the work began), the rule is to fix it before the work ends. A gate tolerated red stops meaning anything. The next contributor cannot tell whether their change broke something, and the self-checking claim dissolves. 'Not mine' is how broken windows accumulate. Only a repository whose sessions leave it cleaner than they found it earns the self-checking label.

The estate, not the web

The web was never the whole problem; the digital estate is. The CMS is one publisher among many. The datalake publishes. The shared drive publishes. The cloud bucket, the API, and the documentation repo each hand files to machines outside the context that produced them.

This is the original document problem, made new and urgent. A PDF gets ingested by another organisation's LLM. A page gets scraped and stored in a vector database. A document travels through three systems before an agent acts on it. By then the CMS is gone, and so is the metadata that lived in the platform's tables. What's left is the file. Older approaches embedded no provenance in the document itself: no authorship, no permissions, no declared scope. The agent guesses the rest. MX is built for that moment, with metadata that travels with the file instead of staying behind. The rule is simple. Wherever a machine reads content away from its origin, that place is a publishing point, and it faces the same exposure. That covers most of what an organisation owns.

The sealed perimeter

The work comes to your data, and your data stays where it is. An MX audit reads what is in front of it: a path on a drive, a removable disk, a folder of static pages, a repository on the device. A website does not have to be live to be assessed, and a repository does not have to be remote; either can be a checkout on a stick. The audit runs inside your perimeter, on a local model, and reaches out only when you authorise it and only for the objects you name. A tool that uploads your estate to assess it has already conceded the argument, whatever its policy says. A tool that works from your own disk, and asks before it reaches out, can be used on material you could never have sent anywhere. That is what makes the evidence usable where it is needed most.

What the machine sees

The audit reads a file as an agent reads it, and it can also meet a page as an agent meets it. A live page is whatever survives the render, the overlays, and the scripts, arranged into the accessibility tree a machine actually perceives, and that can differ sharply from the markup the author shipped. The audit drives a browser as an agent would and captures that tree: the elements it can name, the controls it can reach, and the structure it can use. The owner sees the page as the machine sees it, not as the design intends it.

Reading is only half the exposure. An agent that arrives to do something (find the price, open the navigation, reach the checkout) fails in ways a human visitor would not: a button covered by a consent banner, a control with no accessible name, a menu missing from the tree. The audit reports whether an agent could navigate and act, and where it was blocked. The rule for files holds for pages too. An agent that can read a page but cannot act on it has been served half the contract.

Human in command

Automation runs inside the human's authority and does not replace it. Every automated system in the estate operates under Human-in-Command: a person decides when and how each system is used, and the machine acts only within the output boundary they declared, never beyond it without direction. The principle predates the agents (it is the European

High-Level Expert Group's statement of what keeps an automated system accountable), but agents are what make it bite. A system that can read a file, decide, and act needs an edge it cannot quietly cross.

MX makes that edge explicit and the crossing visible. A file declares its scope: what it is for, how it may be used, what an automated reader may do with it. That scope travels with the file, so an agent that meets the file alone reads the limit of its permission instead of inferring one. The audit already works this way. It runs inside your perimeter and asks before it reaches out. Because the machine's actions are recorded as well as allowed, the human keeps standing authority over the system, far beyond the first decision. The record shows the system stayed inside its boundary, or shows precisely where it did not. Determinism makes that answer reproducible, and Human-in-Command keeps a person answerable for when it is sought and what is done with it.

In our own practice the principle has a working shape: a gestalt of parties, with the human in command (the HIC) alongside the AI parties that draft and build, and any party welcome to join. Any party may draft. Only the human's explicit command routes work to finalisation, and only one authoritative environment finalises it. We run the same discipline we sell.

What MX is

MX is to machines what UX is to users: where UX asks "can a human use this?", MX asks "can a machine understand and act on this?" It is a machine understanding standard rather than a content standard. Content standards govern what you publish. Machine understanding standards govern what machines can reason about, regardless of what was published, where it lives, or which system produced it. Apply the practice to a repository with COGs and the result is a Document Knowledge Graph: provenanced, machine-readable, and traversable by agents. The moonshot: the standard every AI agent reads before it acts, by 2028.

MX is metadata that records a file's provenance, the context that produced it, and how it is meant to be used. The metadata travels with the file. When an agent, a validator, a screen reader, or a parser meets the file alone, the file still states what it is, who it is for, where it belongs, and how it may be reused.

MX adds this where existing standards leave a gap, and never duplicates them. Where Dublin Core, Schema.org, DCAT, EXIF, or the IETF signature standards already cover a need, MX defers. A well-formed MX file is already a well-formed SEO page and an accessible one. It gives every reader, human and machine, the explicit structure each was going to look for anyway.

The two pillars

MX makes content machine-readable. On its own it does not make content trustworthy; REGINALD does that. It attests facts and lets any party verify them, client-side, with their own tools. Read the two together. An agent that can both read a file and verify its claims cites the attested fact instead of inferring one. The effect is fewer hallucinations, lower inference cost, and a record the organisation can show on request. MX supplies readability and REGINALD supplies trust. They do different jobs, so never merge them into one.

The layers, kept distinct

MX is the metadata: a private protocol, created for the MX books and stewarded by CogNovaMX while we run it live as a test. It is published under the MIT licence, so anyone may implement it, and it will be donated to The Gathering as an open standard. COGs (Community Owned Governance Standards) are the containers that carry the metadata across every format. A registry is the public infrastructure that holds, indexes, and serves them. REGINALD is one proprietary registry built on that standard. Everything beneath REGINALD is open to any implementer; the registry CogNovaMX

operates is ours to run well. Collapsing them into a single brand is the common error. Keeping them separate means an operator can adopt the standard without adopting any one vendor.

Built to compound

Most standards fail because adoption requires sacrifice, not because they are wrong. You stop what you are doing, add the metadata, and fill in the schema. The cost lands immediately and the benefit flows elsewhere. That is the compliance model, and it atrophies.

MX is built on a different principle, the one Tim O'Reilly named the architecture of participation: a system where using it and contributing to it are the same act. Linux routes every developer's self-interested fix back into the codebase. Google's PageRank turns every author's self-interested citation into a vote in the index. In both, one act serves the actor and the whole, and nobody is asked to do extra work for anyone else.

When an organisation writes a COG for its own repository, to make its content more useful to its own agents, it also produces a structured, provenanced artefact others can use. One file serves both purposes. REGINALD compounds this: every registration makes the registry more valuable for every agent that queries it, without anyone being recruited to participate.

MX asks organisations only to make their content more useful to their own machines. The shared benefit is a side effect of the structure, designed in from the start. Standards that depend on altruism atrophy, and standards built on self-interest compound.

What MX is not

MX is not compliance in a box. It neither makes an estate safe nor satisfies any regulation on anyone's behalf. Meeting the law stays the organisation's legal duty. What MX produces is evidence: the documentation a regulator expects, made structured, machine-readable, tamper-evident, and verifiable on request.

Regulatory attention is moving upstream, from the behaviour of models to the inputs they are fed. Frameworks such as the EU AI Act and the European Accessibility Act already expect an organisation to produce records of how its systems handle content. MX makes those records queryable, rather than assembled by hand after an inquiry begins. Sold honestly as verifiable evidence, it is what auditors and regulators actually need. Sold as a compliance guarantee, it fails on the first inquiry.

> **Note:** This page describes regulatory frameworks in general terms only. Nothing here is legal advice. Requirements vary by jurisdiction, organisation type, and use case. Consult qualified legal specialists for guidance specific to your situation.

The opportunity, not the threat

This is an opening, not a reckoning. The vendor who adds the machine clause first earns the trust the others are about to lose. A CMS that ships MX metadata by default. A cloud platform that decorates objects on ingest. An agency that audits a client's estate for machine-readiness. Each turns an unsigned obligation into a product. The work is bounded and the standard is open. Waiting costs trust, one wrong agent answer at a time.

What this asks of you

- **A vendor or platform:** ship the second clause. Make the content you publish readable and verifiable to the machine that reads it next.
- **An enterprise:** treat every file as a publishing point. Govern the estate, not only the site.
- **A regulator:** look upstream. The inputs are where the risk begins, and verifiable inputs are where evidence begins.
- **A standards body:** MX defers to your work and fills the gaps you left open. The Gathering keeps it that way, in the open.

In one line

The publishing contract was written for humans. Machines read everything now. MX is the metadata that lets the contract hold for both, and the evidence that proves it did.



This PDF is MX Compatible. It carries machine-readable structure, AI-governance provenance, and the metadata that lets a regulator, an AI agent, or a screen reader walk the document with the same confidence a sighted reader does. Scan the QR code or visit https://mx.allabout.network/manifesto to read what that means.

Tagged PDF · Provenance: EU AI Act · UK ICO · NIST AI RMF